From Probability to Assistant: How Language Models Work and How They Learn to Help

Lecture 2 essay — Outline

I. The Core Idea

- Every modern LM (ChatGPT, Claude, Grok) is one machine: **given some text, guess the next token**. Everything else — math, architecture, training — is machinery for making that guess well
- Four movements: (1) the model as a probability engine, (2) text → math, (3) math → text, (4) raw predictor → assistant

II. Movement I — The Model Is a Probability Engine

- **Autoregressive** generation: compute $P(x_t \mid x_1, \ldots, x_{t-1})$, append the chosen token, repeat. Every reply is built one token at a time
- **Attention** ("Attention Is All You Need," 2017) is the architectural breakthrough: the model weights which earlier tokens matter for the current guess. Resolving "it" in the trophy/suitcase sentence is attention at work
- A **probability distribution** = every possible outcome with a likelihood, satisfying $\sum_i P(x_i) = 1$ and $P(x_i) \geq 0$ (fair die: $P(i) = \frac{1}{6}$). At every step the model outputs exactly this object, over its whole vocabulary
- **Sampling** = drawing one outcome from that distribution (rolling the die). This is why the same prompt gives different answers — nothing is retrieved; randomness is built into the draw
- **Generative modeling** = learning the distribution from data rather than writing it down. For sequential data the question is always conditional: next event given the sequence so far
- **Chain rule**: the joint factors into conditionals,

$$P(x_1, \ldots, x_n) = P(x_1)\, P(x_2 \mid x_1) \cdots P(x_n \mid x_1, \ldots, x_{n-1}) = \prod_{t=1}^{n} P(x_t \mid x_{<t}),$$

  so token-by-token sampling is mathematically identical to sampling the whole sequence at once — the loop is a faithful implementation, not a hack. The two-variable case $P(x, y) = P(x \mid y)\, P(y)$ gives Bayes' rule by symmetry:

$$P(y \mid x) = \frac{P(x \mid y)\, P(y)}{P(x)}$$

- **Models mirror their training data**: train on Austen, write like Austen; train on the internet, learn the distribution of human text — knowledge, styles, and biases included
- **Emergence**: optimizing only next-token prediction yields untaught abilities (translation, arithmetic, rudimentary reasoning). Flip side is **overfitting**: too little data → memorizing, not generalizing
- **Steering**: at inference there is one wheel — the prompt. Prompt engineering = choosing a beginning whose most probable continuations are the ones you want

III. Movement II — How Text Becomes Math

- **Tokenization** (typically byte-pair encoding) chops text into sub-word chunks mapped to IDs. Explains two quirks: context windows are measured in tokens, and models miscount letters in "strawberry" because they see IDs, not characters
- **One-hot encoding**: token k becomes $e_k = [0, \ldots, 1, \ldots, 0] \in \mathbb{R}^V$ with a single 1 at position k. Works, but is huge and encodes no relationships — all one-hot vectors are orthogonal ($e_i \cdot e_j = 0$ for $i \neq j$), so "cat" is as far from "kitten" as from "carburetor." Best understood as the problem embeddings solve
- **Embeddings**: each token → a learned point $v \in \mathbb{R}^d$, $d \approx 1000$–3000; related words land near each other, and directions carry relationships:

$$v_{\text{king}} - v_{\text{man}} + v_{\text{woman}} \approx v_{\text{queen}}, \qquad \text{equivalently} \qquad v_{\text{king}} - v_{\text{queen}} \approx v_{\text{man}} - v_{\text{woman}}.$$

  Meaning becomes geometry

- **Positional embeddings** inject order (absolute or relative), since "the cat chased the dog" and its reverse share identical tokens
- **Transformer**: a stack of repeated blocks, each an attention layer (tokens exchange information by learned relevance) plus a feedforward layer (per-position processing). Each token issues a query Q, is indexed by a key K, and carries a value V:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

IV. Movement III — How Math Becomes Text Again

- **Logits** $z_1, \ldots, z_V$: the transformer's raw per-token scores — can be negative, don't sum to 1; the model's unnormalized opinion
- **Softmax** exponentiates and normalizes, giving a genuine distribution (back to Movement I):

$$P(x_i) = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}$$

- **Temperature** divides logits by T before softmax:

$$P(x_i) = \frac{e^{z_i/T}}{\sum_{j=1}^{V} e^{z_j/T}}$$

  As $T \to 0$ the distribution collapses onto the top token (greedy); as T grows it flattens toward uniform. Low = predictable and conservative; high = more varied and more error-prone. A literal API parameter, not a metaphor
- **Decoding**: greedy takes $x_t = \arg\max_i P(x_i)$ — deterministic but dull and repetitive; sampling draws $x_t \sim P$ — variety at the cost of occasional bad draws
- The **"NPRD" glitch**: one low-probability draw produces a garbled token, but because each later prediction conditions on the full context (chain rule again), the model absorbs the error and steers back to coherence
- **The full loop**: tokenize → embed → position (II); transformer → logits → softmax → sample (III); append and repeat (I). Every sentence a model has ever produced is this loop

V. Movement IV — From Raw Predictor to Assistant

- A pure next-token predictor is not an assistant: ask a question and it may continue with more questions, because on the internet questions come in lists
- **Entropy** measures a distribution's uncertainty (sum of two dice piles near 7 → lower entropy than one uniform die):

$$H(p) = -\sum_i p(x_i) \log p(x_i)$$

- **Cross-entropy** compares a predicted distribution q to the true one p — and it *is* the training loss:

$$H(p, q) = -\sum_i p(x_i) \log q(x_i)$$

  With all true mass on the observed token, the per-step loss reduces to $-\log q(x_t^*)$, the negative log-probability assigned to the correct token. Every pre-training gradient step lowers this
- **Three-stage pipeline**:
  - **Pre-training** — internet-scale next-token prediction; the source of essentially all knowledge and capability
  - **Supervised fine-tuning (SFT)** — human-written demonstrations of good answers; teaches the *format* of being an assistant
  - **RLHF** — humans rank output pairs, a reward model $r(x, y)$ learns the rankings, and the policy π_θ is optimized for reward with a KL penalty keeping it near the SFT model:

$$\max_\theta \; \mathbb{E}_{y \sim \pi_\theta}[r(x, y)] - \beta \, D_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{SFT}})$$

- **Human side**: rankings and demonstrations come from large labeler workforces; pay, conditions, and content exposure are a live ethical issue
- **Base vs. aligned**: base = raw continuer (no persona, refusals, or safety behavior); aligned = base + SFT + RLHF. The asymmetry — capability in the base, safety layered on after — is *why jailbreaks work*: a prompt teaches nothing new, it routes around the layer
- **Superficial alignment hypothesis**: all knowledge and ability come from pre-training; alignment only chooses what to surface and in what style. If true, safety is a thin veneer — easier to strip than to build. Whether it holds in full is open
- **DPO** reaches RLHF-like results with no separate reward model, working directly from preference pairs — "your language model is secretly a reward model." Widely adopted in open source
- **Cut for time**: LLM reasoning (chain-of-thought, dedicated reasoning models) — currently the most active research area; recording to follow

VI. Four Movements, One Machine

- Probability engine (learn a distribution, sample from it, chain rule makes this principled) → text becomes math → math becomes text → alignment layered on top, with honest open questions about how deep that layer goes
- Every chatbot conversation is this entire stack executing once per token. The magic is that "guess the next token, well enough, at scale" contains so much

Quick review

What's strong: The four-movement structure is a real pedagogical device, not decoration — the loop closes (I → II → III → I), so the "one machine" claim is earned structurally rather than asserted. The chain-rule point (token-by-token sampling is a faithful implementation, not a hack) is the single most important idea for a non-specialist reader and it is placed exactly where it pays off, then reused twice (NPRD recovery, the closing synthesis). The base/aligned asymmetry → jailbreak explanation is the cleanest bridge in the essay from mechanics to policy, and "cross-entropy is not related to training, it *is* the loss" is the right emphasis.

Where to push: (1) The essay states "capability lives in the base model, safety is a layer added afterward" as settled, then introduces the superficial alignment hypothesis as an open question — but the first claim is most of the hypothesis. Worth separating the well-supported version (post-training changes behavior far more than it adds knowledge) from the strong version. (2) The picture is pre-2024: reasoning models trained with RL on verifiable rewards do appear to add capability in post-training, not just style, which is exactly the seam the cut "reasoning" lecture will have to revisit. (3) Emergence is presented uncritically; there is a live debate that some "emergent" jumps are artifacts of discontinuous metrics rather than discontinuous ability. (4) The labeler workforce is framed only as a labor-ethics issue; it is also an epistemic one — the reward model $r(x, y)$ encodes *whose* preferences define "helpful," which matters for any downstream claim about what an aligned model "values." (5) Attention is introduced in Movement I as the breakthrough and only given its $Q/K/V$ form in Movement II; the reader should know these are the same object. (6) Minor: one-hot vs. embedding is presented as problem/solution, but an embedding layer is literally $e_k^\top W$ for a learned matrix W — a lookup table — which is a useful demystification.

Connection to your work: The base/aligned asymmetry is the load-bearing fact for governance. If safety behavior is a removable layer, then rules that mandate behavior at the deployment surface are regulating the veneer, while capability sits upstream in pre-training and compute — which is the structural argument behind the "regulate training runs vs. regulate deployments" split. Two smaller hooks: "models mirror their training data" is the technical premise of every training-data provenance and copyright dispute; and sampling means outputs are non-deterministic by design, which bears on reproducibility of AI-generated evidence and on any test that asks what a model "would have said."